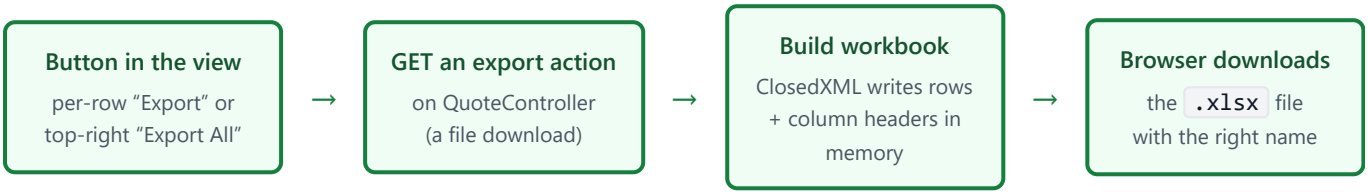
QUOTESPROJECT — BUILD GUIDE • EXCEL EXPORT

Exporting Quotes to Excel

A per-quote export (header + its lines → *quote (number).xlsx*) and an "Export All" export (every quote's header fields → one sheet), using the free **ClosedXML** library.

0. What you're building



Two exports, both returning a real `.xlsx` the browser downloads:

- **Export one quote** — the quote's header fields *and* its lines, in a file named `quote <QuoteNumber>.xlsx`.
- **Export all** — every quote's header fields as rows in a single sheet.

> **Why ClosedXML**
>
> `ClosedXML` is free (MIT licence), reads/writes real `.xlsx` files, and has a clean API. The other popular option, EPPlus, now needs a paid licence for commercial use — so ClosedXML is the better fit here. One NuGet package, no Excel install required on the server.

The mechanics of a file download in ASP.NET Core

An export action just returns `File(bytes, contentType, fileName)`. That sets a `Content-Disposition: attachment` header, which tells the browser "save this, don't display it" and gives it the file name. Because it's a plain **GET**, the button can simply navigate to the URL — no `fetch`/blob juggling — and your auth cookie is sent automatically. The MIME type for `.xlsx` is `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`.

Step 1 — Install ClosedXML

`Package Manager Console (or the NuGet UI)`

```
dotnet add package ClosedXML
```

That's the only dependency. It pulls in the OpenXML bits it needs.

Step 2 — Give QuoteController access to quote lines

Your `QuoteController` currently injects only `IQuoteService`. The per-quote export also needs the lines, so add `IQuoteLineService` to the constructor — exactly as `QuoteLineController` already does.

`Controllers/QuoteController.cs`

```
private readonly IQuoteService _quoteService;
private readonly IQuoteLineService _quoteLineService;   // ← add

public QuoteController(IQuoteService quoteService, IQuoteLineService quoteLineService)
{
    _quoteService = quoteService;
    _quoteLineService = quoteLineService;                // ← add
}
```

No change to `Program.cs` — `IQuoteLineService` is already registered.

Step 3 — Export one quote (header + lines)

`Controllers/QuoteController.cs  (new action)`

```
using ClosedXML.Excel;
using Microsoft.AspNetCore.Authorization;

[HttpGet("export/{quoteId:int}")]     // GET /Quote/export/5
[Authorize]                            // any logged-in user may export (change to Roles="Admin"
to restrict)
public IActionResult ExportQuote(int quoteId)
{
    Quote quote;
    List<QuoteLine> lines;

    try
    {
        quote = _quoteService.GetQuoteById(quoteId);       // your existing service
        lines = _quoteLineService.GetQuoteLines(quoteId);
    }
    catch (KeyNotFoundException)
    {
        return NotFound("Quote not found.");
    }

    using var workbook = new XLWorkbook();
    var ws = workbook.Worksheets.Add("Quote");

    // --- Header block (label in col A, value in col B) ---
    ws.Cell("A1").Value = "QUOTE";
    ws.Cell("A1").Style.Font.Bold = true;
    ws.Cell("A1").Style.Font.FontSize = 16;

    ws.Cell("A3").Value = "Quote Number";    ws.Cell("B3").Value = quote.QuoteNumber ?? "";
    ws.Cell("A4").Value = "Customer";        ws.Cell("B4").Value = quote.Customer ?? "";
    ws.Cell("A5").Value = "Address";         ws.Cell("B5").Value = quote.Address ?? "";
    ws.Cell("A6").Value = "External Order #"; ws.Cell("B6").Value = quote.ExternalOrderNumber ??
"";
    ws.Cell("A7").Value = "Order Date";      ws.Cell("B7").Value =
quote.OrderDate?.ToString("yyyy-MM-dd") ?? "";
    ws.Cell("A8").Value = "Due Date";        ws.Cell("B8").Value = quote.DueDate?.ToString("yyyy-
MM-dd") ?? "";
    ws.Cell("A9").Value = "Invoice Date";    ws.Cell("B9").Value =
quote.InvoiceDate?.ToString("yyyy-MM-dd") ?? "";
    ws.Range("A3:A9").Style.Font.Bold = true;

    // --- Lines table (starts a couple of rows down) ---
    int headerRow = 11;
    ws.Cell(headerRow, 1).Value = "Item";
    ws.Cell(headerRow, 2).Value = "Quantity";
    ws.Cell(headerRow, 3).Value = "Price";
    ws.Cell(headerRow, 4).Value = "Discount";
    ws.Cell(headerRow, 5).Value = "Line Total";
    var headerRange = ws.Range(headerRow, 1, headerRow, 5);
    headerRange.Style.Font.Bold = true;
    headerRange.Style.Fill.BackgroundColor = XLColor.LightGray;
```

```
    int r = headerRow + 1;
    foreach (var line in lines)
    {
        ws.Cell(r, 1).Value = line.Item ?? "";
        ws.Cell(r, 2).Value = line.Quantity;
        ws.Cell(r, 3).Value = line.Price;
        ws.Cell(r, 4).Value = line.Discount;
        ws.Cell(r, 5).Value = line.Quantity * line.Price - line.Discount;   // adjust to your
discount meaning
        r++;
    }

    ws.Columns().AdjustToContents();    // auto-fit column widths

    // --- Save to a memory stream and return as a download ---
    using var stream = new MemoryStream();
    workbook.SaveAs(stream);

    return File(
        stream.ToArray(),
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        $"quote {quote.QuoteNumber}.xlsx");      // e.g. "quote Q1001.xlsx"
}
```

Read it top to bottom: load the quote and its lines from your existing services → create a workbook and one worksheet → write the header fields as label/value pairs → write the lines as a table with a bold header row → auto-fit → save into a `MemoryStream` and hand the bytes to `File(...)`, which becomes the download. The file name uses the quote number, so it downloads as *quote Q1001.xlsx*.

Two small things to match to your data

Line Total is computed as `Quantity × Price − Discount` — if your `Discount` is a *percentage* rather than an amount, change it to `Quantity × Price × (1 − Discount)`. And `?? ""` guards nullable strings/dates so ClosedXML never receives a null value.

Step 4 — Export all quote headers

Controllers/QuoteController.cs (new action)

```
[HttpGet("exportall")]                // GET /Quote/exportall
[Authorize]
public IActionResult ExportAllQuotes()
{
    var quotes = _quoteService.GetQuotes();   // same source as the list page

    using var workbook = new XLWorkbook();
    var ws = workbook.Worksheets.Add("All Quotes");

    // --- Header row (the column names) ---
    string[] headers = { "Quote Number", "Customer", "Address", "External Order #",
                         "Order Date", "Due Date", "Invoice Date" };
    for (int c = 0; c < headers.Length; c++)
        ws.Cell(1, c + 1).Value = headers[c];

    var headerRange = ws.Range(1, 1, 1, headers.Length);
    headerRange.Style.Font.Bold = true;
    headerRange.Style.Fill.BackgroundColor = XLColor.LightGray;

    // --- One row per quote ---
    int r = 2;
    foreach (var q in quotes)
    {
        ws.Cell(r, 1).Value = q.QuoteNumber ?? "";
        ws.Cell(r, 2).Value = q.Customer ?? "";
        ws.Cell(r, 3).Value = q.Address ?? "";
        ws.Cell(r, 4).Value = q.ExternalOrderNumber ?? "";
        ws.Cell(r, 5).Value = q.OrderDate?.ToString("yyyy-MM-dd") ?? "";
        ws.Cell(r, 6).Value = q.DueDate?.ToString("yyyy-MM-dd") ?? "";
        ws.Cell(r, 7).Value = q.InvoiceDate?.ToString("yyyy-MM-dd") ?? "";
        r++;
    }

    ws.Columns().AdjustToContents();

    using var stream = new MemoryStream();
    workbook.SaveAs(stream);

    return File(
        stream.ToArray(),
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        "All Quotes.xlsx");
}
```

> **Heads-up on `GetQuotes()`**
>
> This exports exactly what the list page shows, because it uses the same `GetQuotes()` method. If that method is capped (yours currently loops a fixed count), the export inherits the same cap — fine for now, just know the two always match.

Step 5 — The buttons in the view

Both are plain GET downloads, so the button just points the browser at the URL — no fetch needed.

(a) Per-row Export — add next to Save / Delete / View in the Actions cell:

`Views/Quote/Index.cshtml  (inside the Actions <td>)`

```
<button class="btn btn-sm btn-outline-success" onclick="exportQuote(this)">Export</button>
```

(b) Export All — add to the top row next to the heading / Create User button:

```
<button class="btn btn-outline-success"
        onclick="window.location.href='/Quote/exportall'">
    Export All
</button>
```

(c) The one helper — add to the page's `<script>`:

```
function exportQuote(btn) {
    const id = btn.closest("tr").dataset.id;    // the row's data-id = QuoteId
    window.location.href = `/Quote/export/${id}`;   // triggers the download
}
```

Setting `window.location.href` to a download URL starts the download *without* navigating away — the page stays put because the response is an attachment, not a page.

Step 6 — Column names (quick reference)

Export	Columns, in order
Single quote — header block	Quote Number, Customer, Address, External Order #, Order Date, Due Date, Invoice Date
Single quote — lines table	Item, Quantity, Price, Discount, Line Total
Export All	Quote Number, Customer, Address, External Order #, Order Date, Due Date, Invoice Date

Step 7 — Test it

Do this	You should see
1. On a quote row, click **Export**	A file *quote <number>.xlsx* downloads; opening it shows the header block and the lines table with the columns above.
2. Click **Export All** (top right)	*All Quotes.xlsx* downloads with one row per quote and the seven header columns.
3. Export a quote that has no lines	Header block still exports; the lines table shows just the header row. No error.
4. Open both files in Excel	Columns are auto-sized, header rows bold, dates readable.

Done

Two real Excel exports with correct column names, driven by your existing services. The pattern — *load data → build an* `XLWorkbook` *→ return* `File(...)` — is the same for any future export (quote lines only, a filtered set, a summary sheet). To split the single-quote export into two worksheets instead of one, just call `workbook.Worksheets.Add("Lines")` and write the lines there.